

Data Lake Without a Data Platform

Real-Time Aggregation at 1,500 messages/sec



What if you need Data Lake behaviour — but without the platform?





About Me

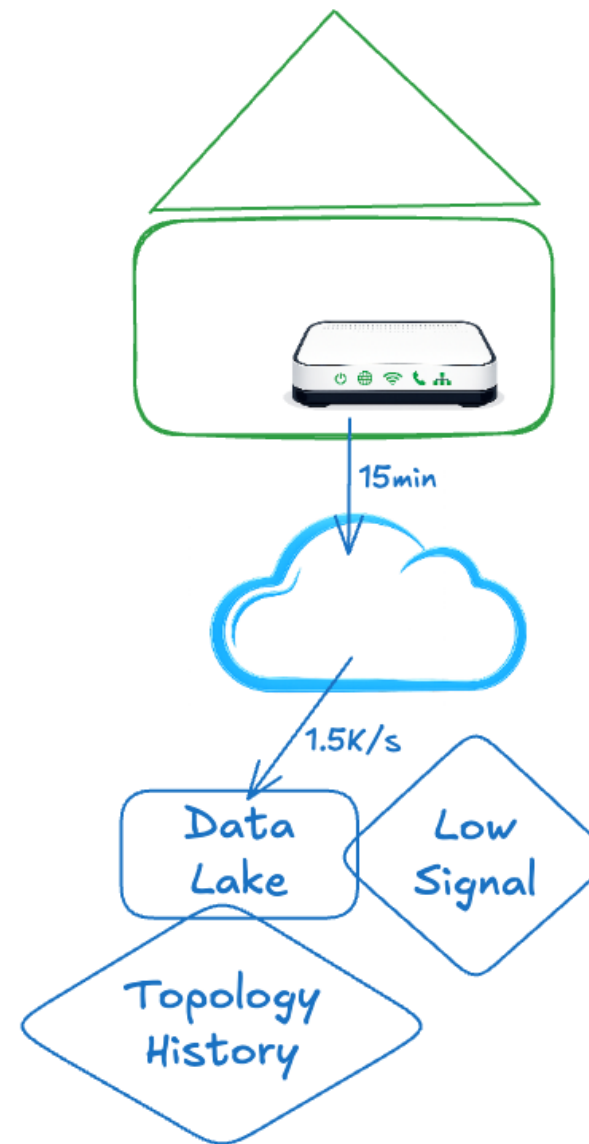
- Distributed systems — designing and operating high-throughput, event-driven architectures in production
- Engineering Manager — leading backend teams, balancing technical direction with delivery
- Telecommunications domain — building data pipelines for telco device telemetry at scale
- Founder of Hypercube

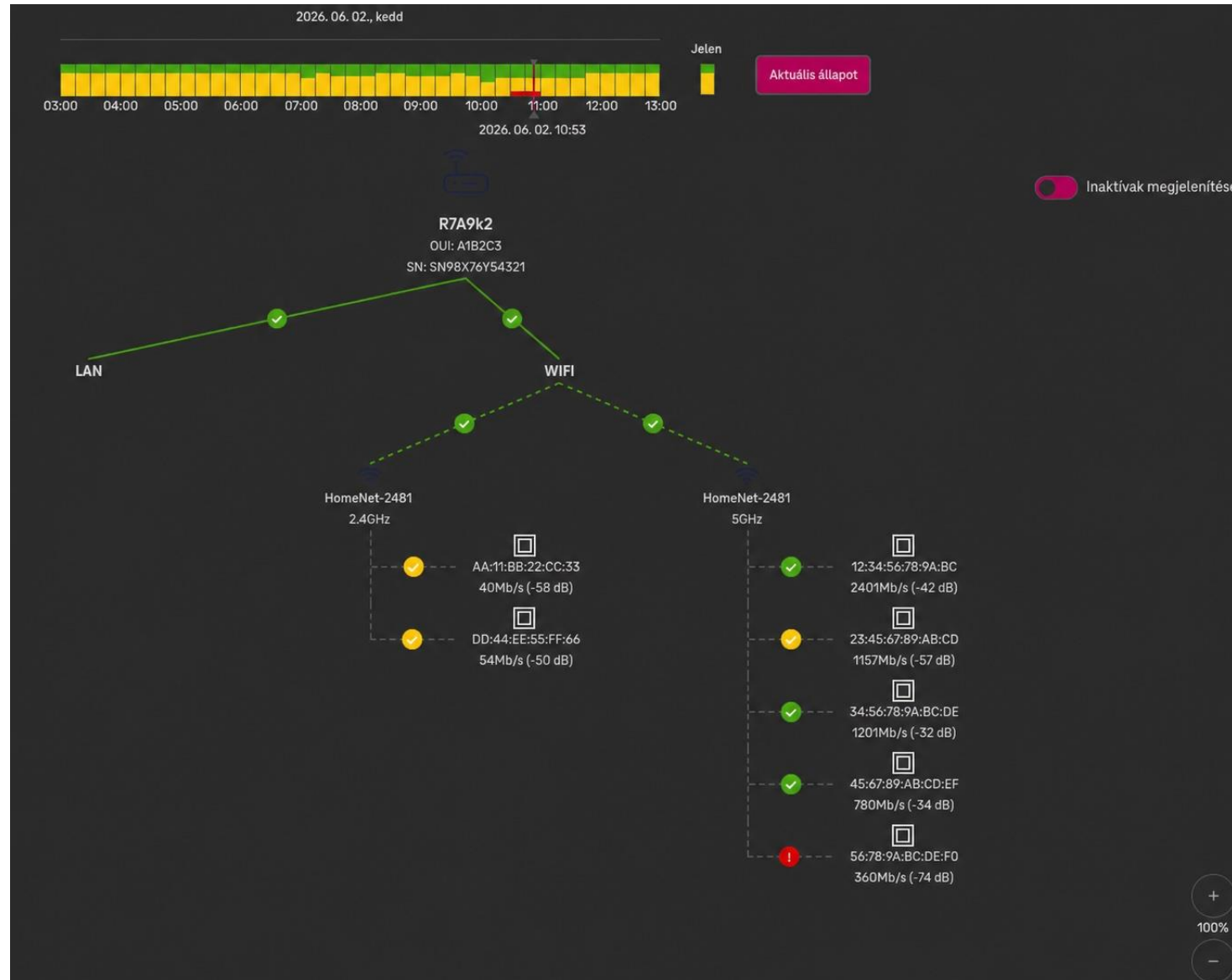
Business Requirement

The challenge: aggregate telemetry from dozens of home gateways — in near real-time.

The constraints:

- ~1.5M devices in homes
- Home gateway data on every 15 minutes
- Raw event retention: 72 hours
- Pattern detection window: 48 hours
- The business needs to know:
 - *Is this device experiencing low signal?*
 - *How looks like the network topology in the past?*





Bejelentések kezelése

Észlelt hiba:

Optikai jelszint kritikusan alacsony

Mit jelent ez a hiba?

Méréseink alapján a vonali kapcsolat rendszeresen megszakad.

Egyeztess az ügyféllel, hogy ő szokta-e rendszeresen újraindítani a modemet!

Ha nem ő szokta újraindítani, indítsd el a DIY eszközcserét Merlin-ben!

Ha az ügyfél szokta újraindítani, akkor az adatok alapján szükség lehet egy helyszíni kiszállásra. Rögzíts hibajegyet a Merlin-ben!

Otthoni hibaelhárítás

WiFi hibaelhárítás

TV hibaelhárítás

HGW újraindítása



First Tries

Attempt 1 • Apache Spark

- The technically correct choice. Kafka → Spark → MongoDB.
- **What went wrong:**
- Operational overhead: constant tuning of memory.
- Resource contention: running Spark on the same infra as the application caused unpredictable failures
- **No metrics.**

First Tries

Attempt 2 · Scheduled Aggregation Jobs

- Simpler: Kafka → MongoDB raw collection → batch job every 60 seconds.

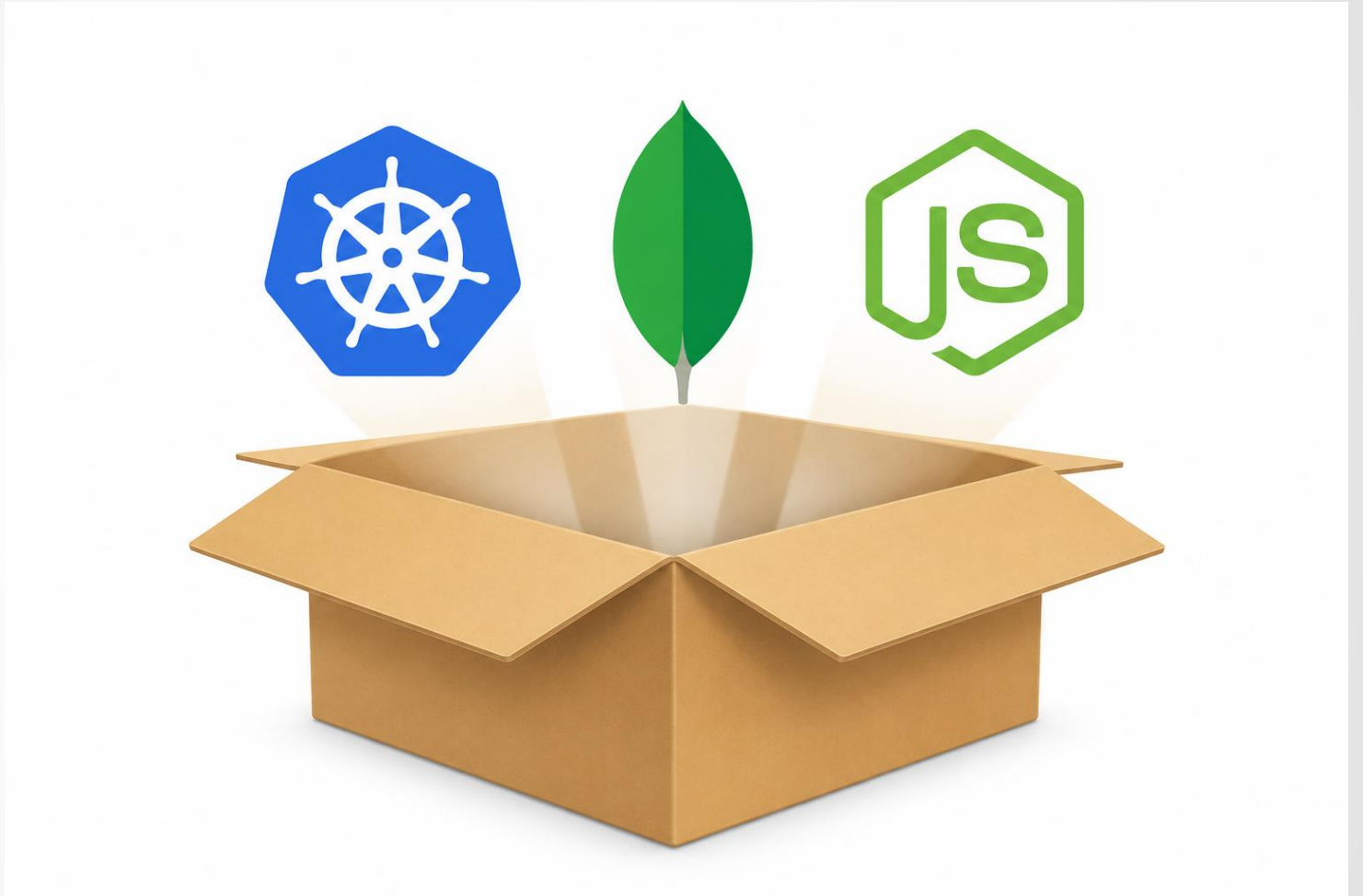
What went wrong:

- Aggregation throughput < ingestion throughput — the queue never caught up
- Lag grew from minutes to hours
- **Again: no monitoring.**



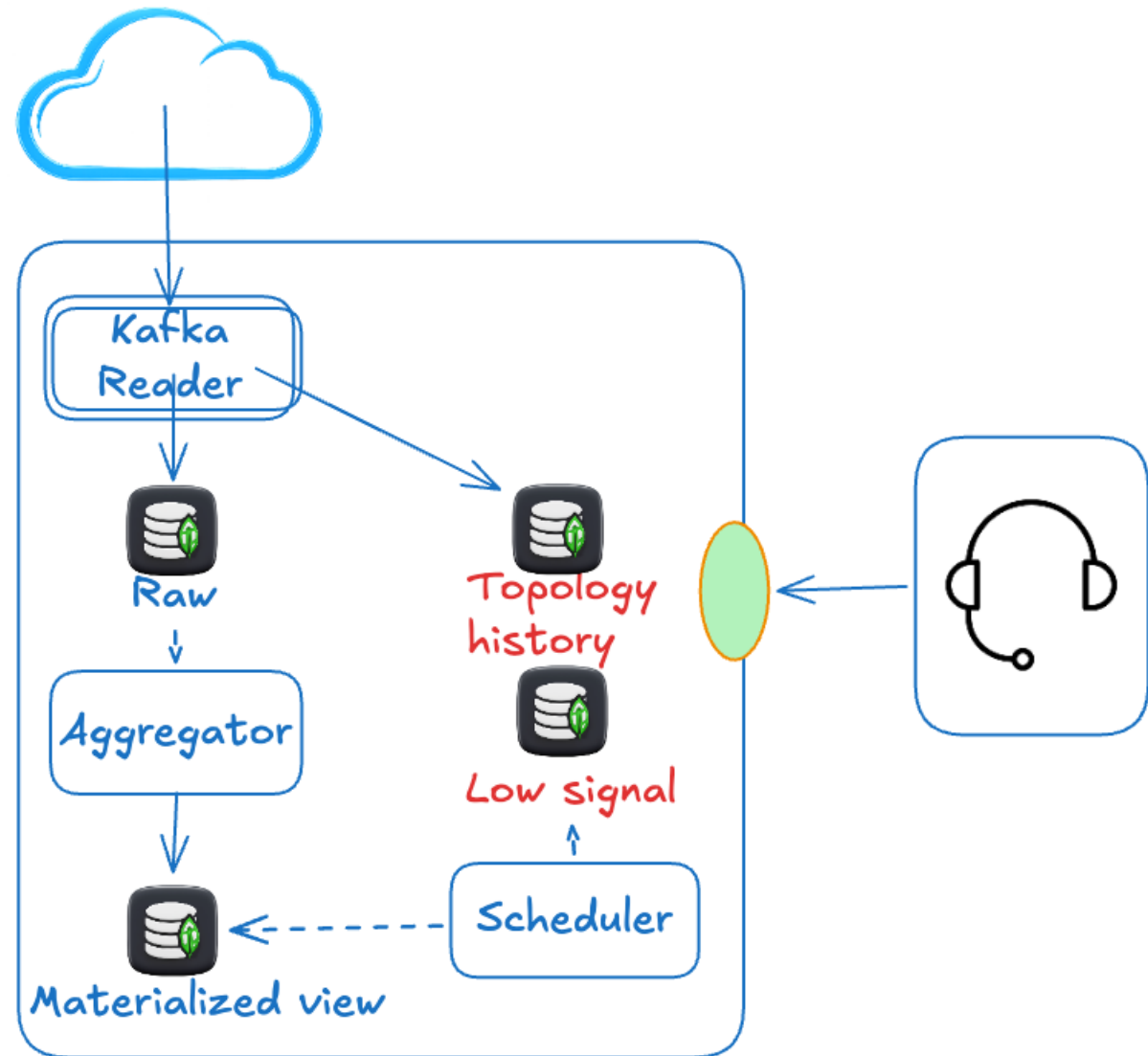
What's in the Box?

Layer	Technology
Orchestration	Kubernetes
Message stream	Apache Kafka
Storage	MongoDB (replica set)
Event processing	MongoDB Change Streams
Application	Node.js
Observability	Prometheus + Grafana



Solution

- Change stream with resume token
- Micro-batch
- Single processor instance to aggregate data
- Scheduled jobs

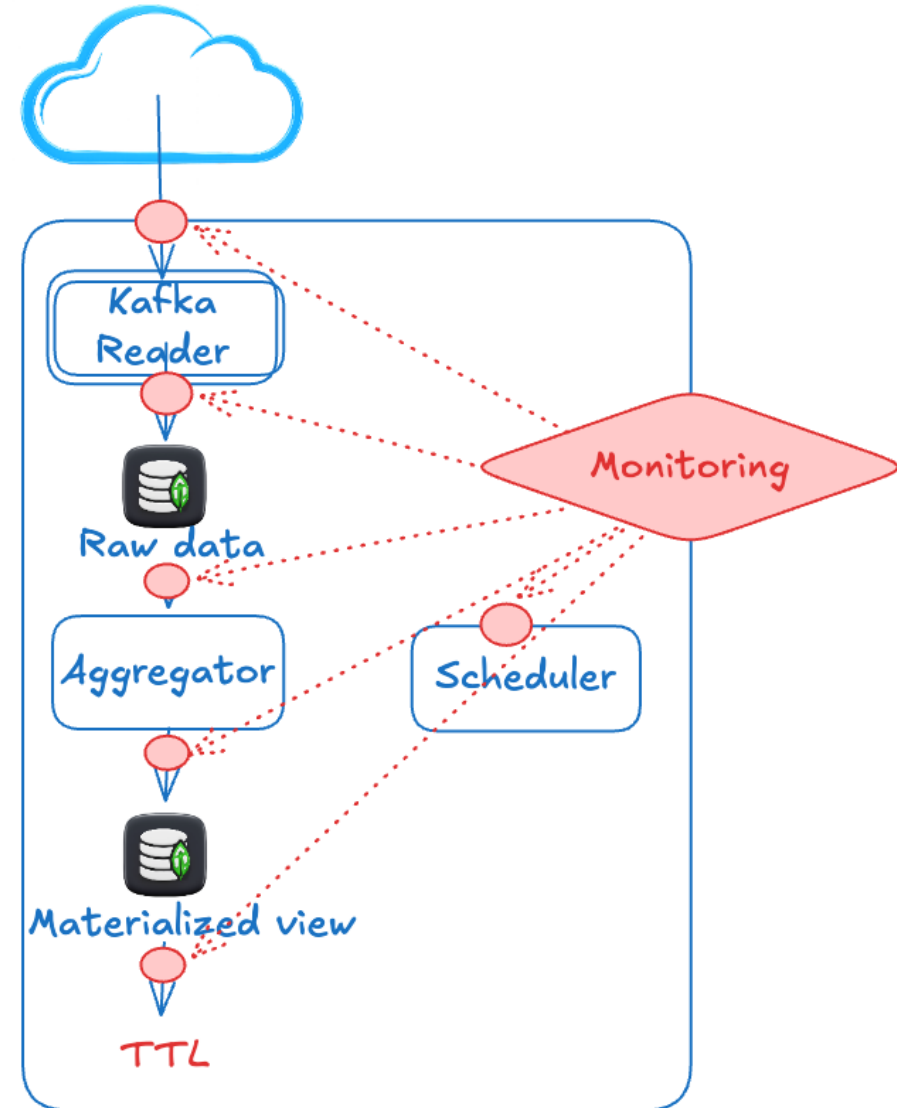


Monitoring

” **Visibility is not optional.
It's the architecture.**”

Every element of the pipeline needs its own monitoring and alerts.

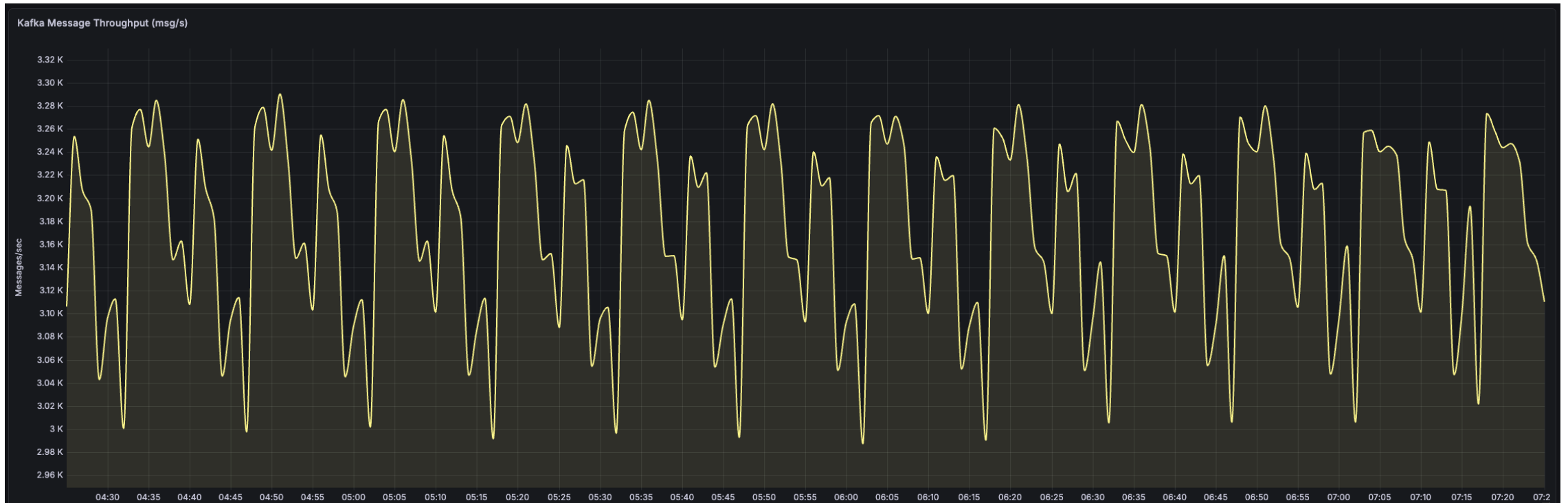
- Events/sec · Event lag - are we keeping up with the producer?
- Batch size · Flush frequency - how full is the micro-batch window?
- Change stream lag - time between INSERT and aggregation write



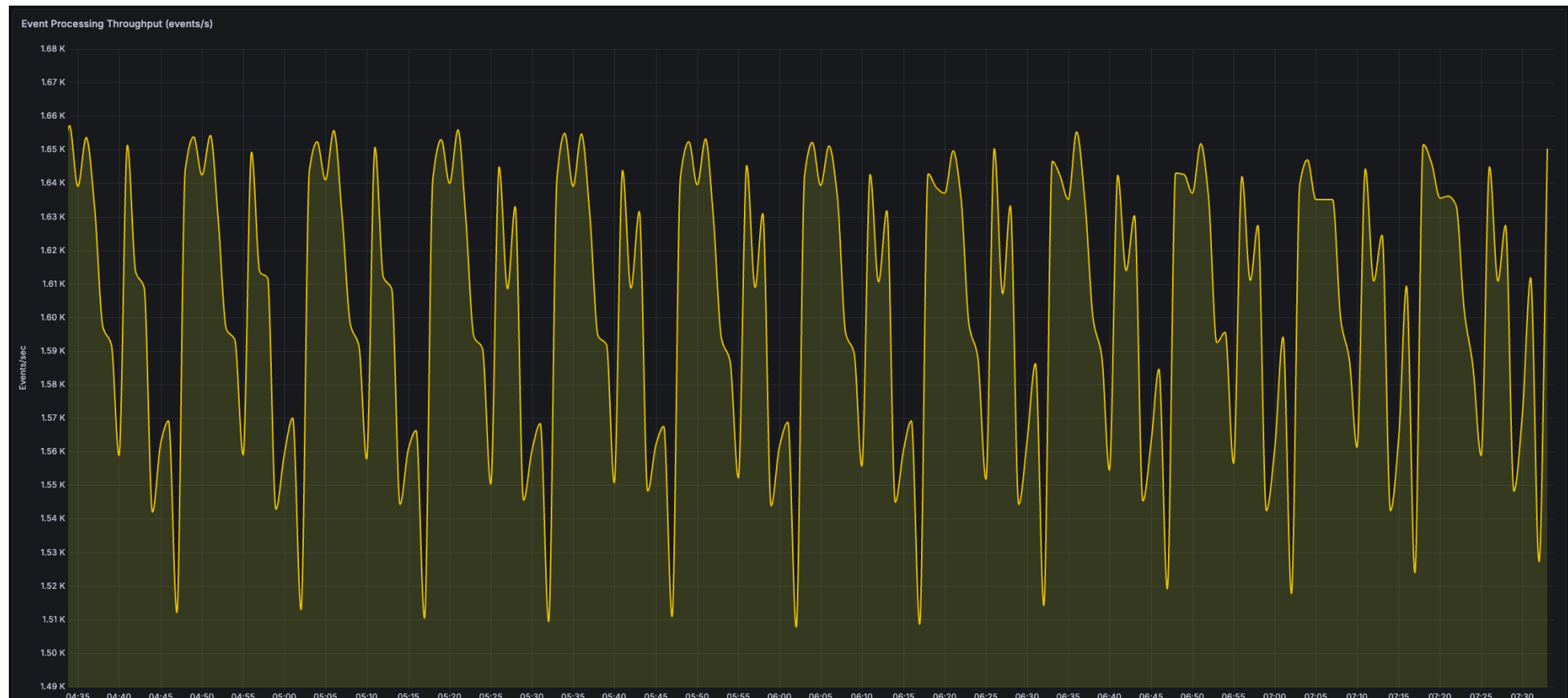
Kafka throughput



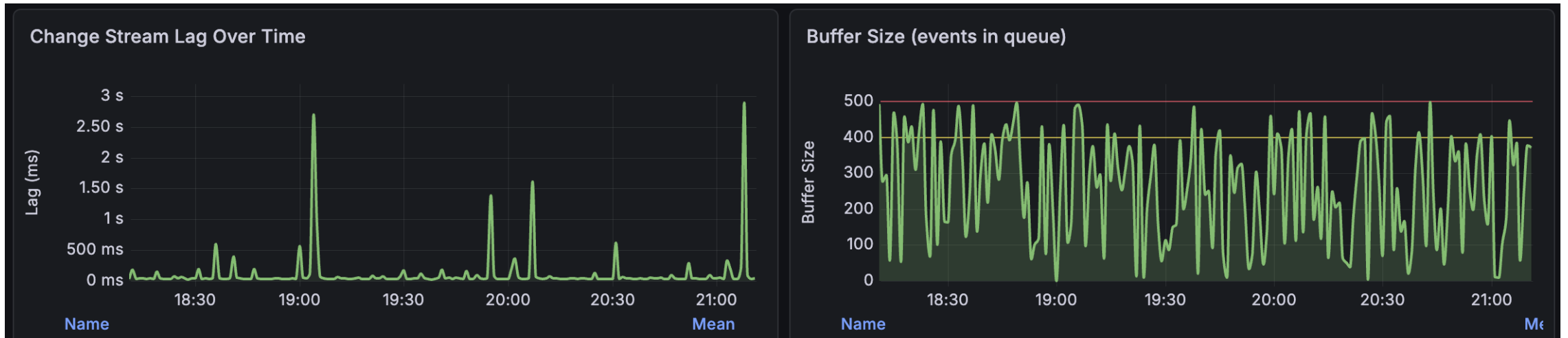
Kafka reading



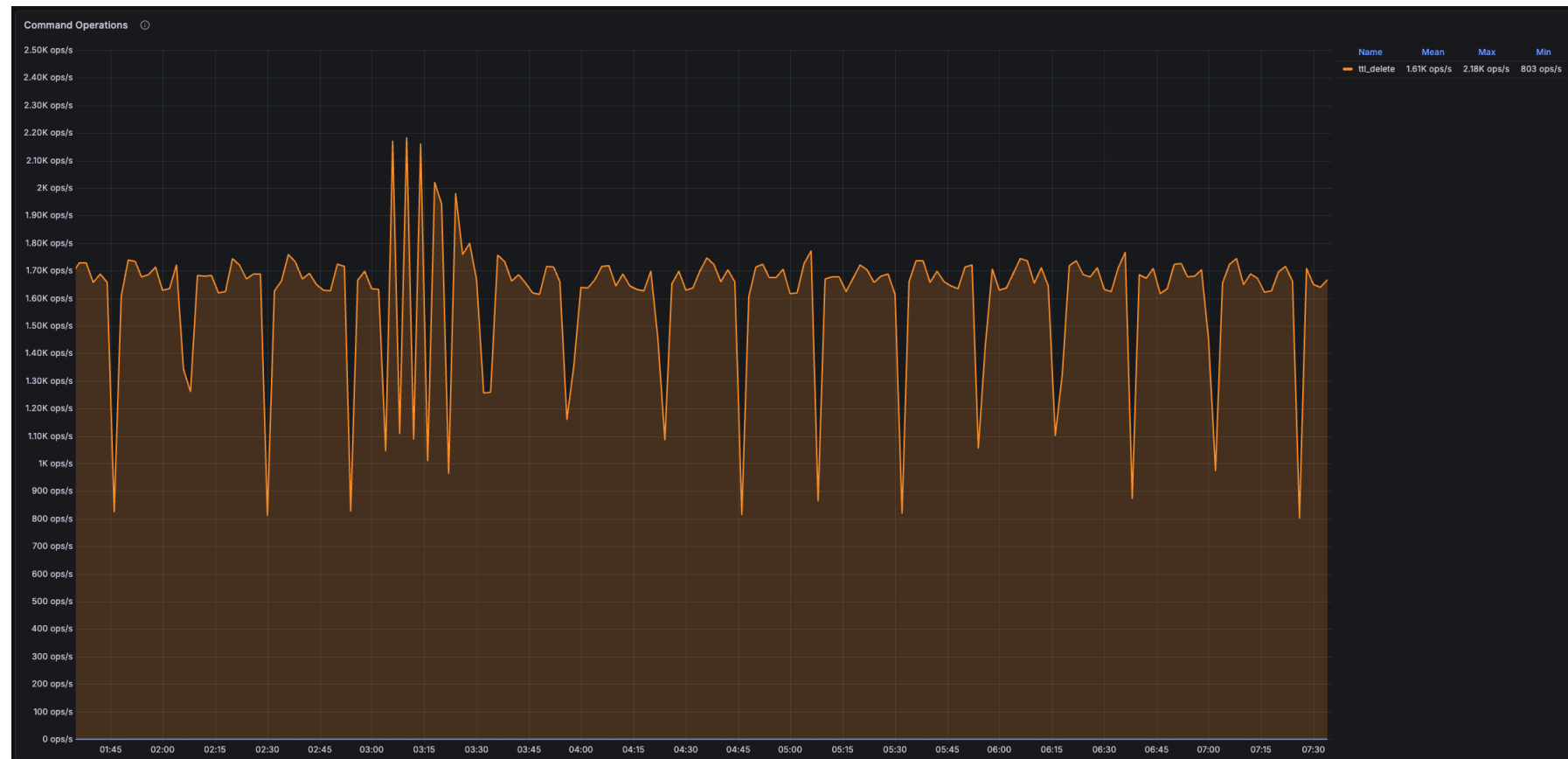
MongoDB Change Stream



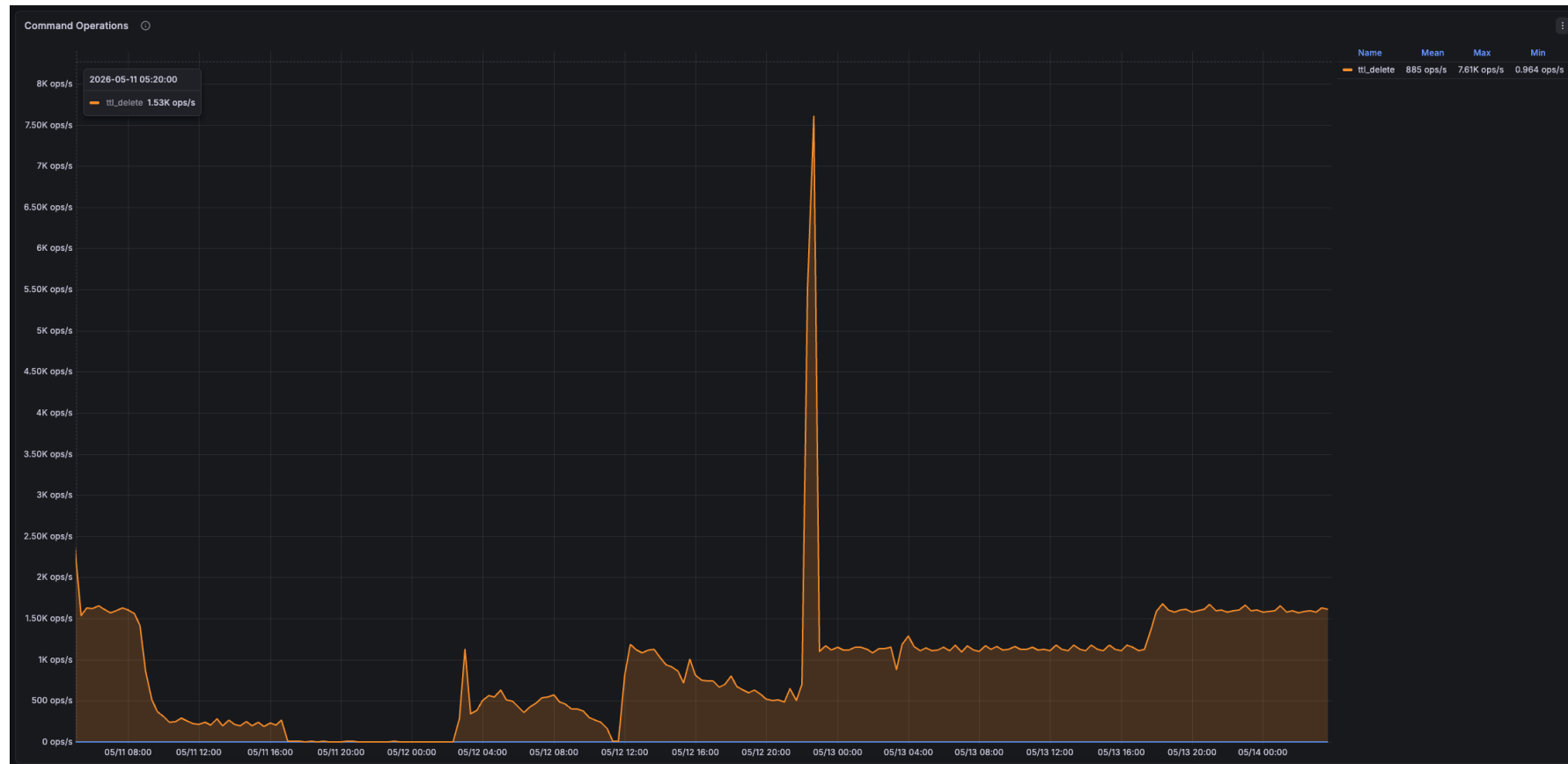
Change Stream lag and buffer



TTL



TTL / broken



Thank You —
Questions
Welcome over a
Beer 🍺



@medium